

AI + Claude + Go + LLM + Programming +

Effective Go Development with Claude: Best Practices for AI Pair Programming

Davin Hills

Follow

5 min read · Jul 14, 2025

👍 12

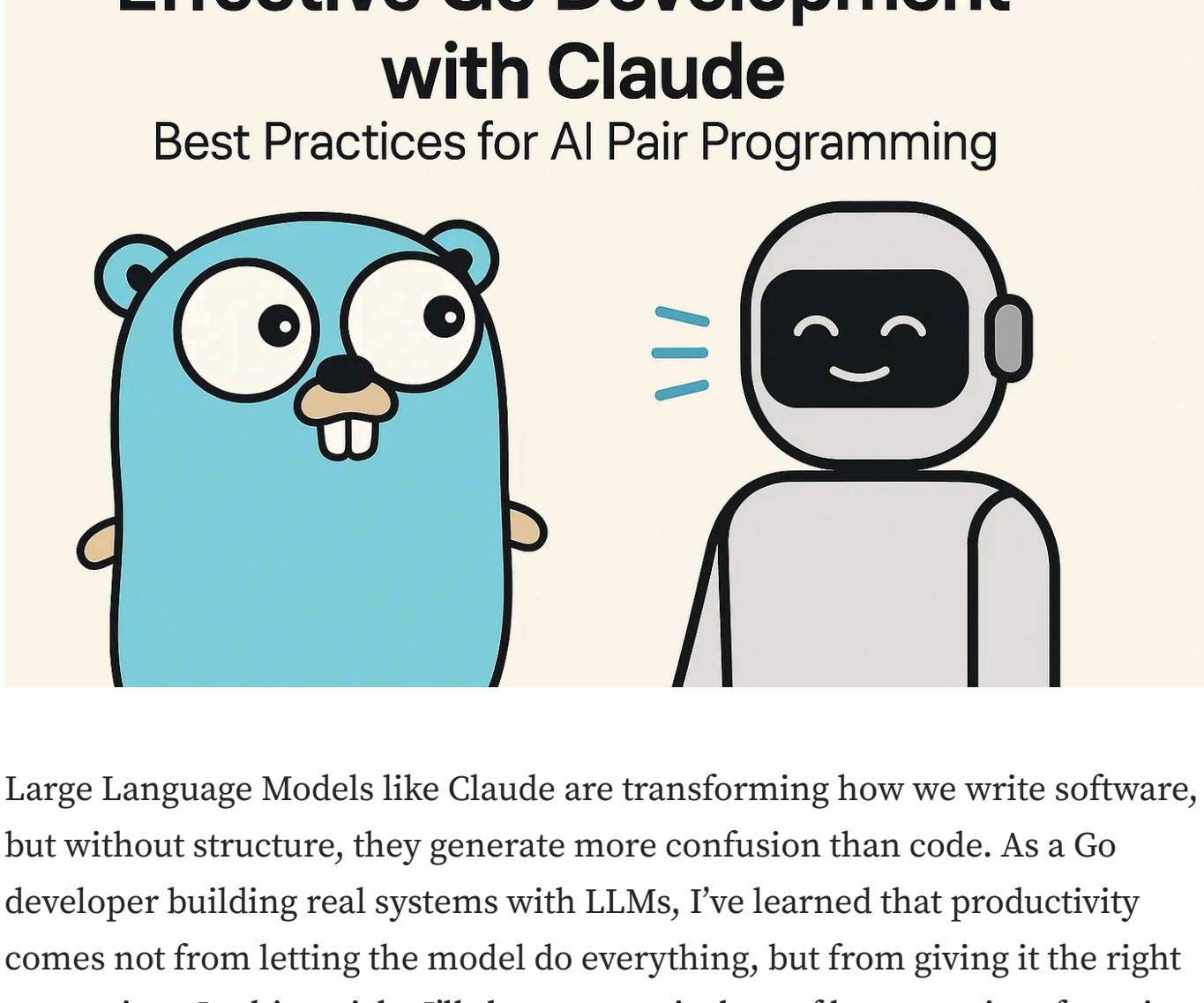
💬 4

🔖 1

🔗

🔄

📄



Large Language Models like Claude are transforming how we write software, but without structure, they generate more confusion than code. As a Go developer building real systems with LLMs, I've learned that productivity comes not from letting the model do everything, but from giving it the right constraints. In this article, I'll share a practical set of best practices for using Claude to write Go code effectively: how to structure your project, enforce code quality, leverage sub-agents, and build a collaborative workflow that makes AI a reliable pair programmer, not a wildcard.

📁 I. Setup and Foundation

1. Define Your Directory Structure Early

Claude can infer quite a lot from a project's layout — *if* it has one.

Setting up your ``cmd/'`, ``pkg/'`, ``internal/'`, ``api/'`, and ``scripts/'` folders early tells Claude where different pieces of the architecture belong. It doesn't need to invent structure if you show it. A good directory tree acts like a GPS — Claude sees it and knows where to go.

Even better: reference the directory structure in prompts. Say `"create a handler in `internal/handlers/"` using the interface in `"pkg/user"` and it will just work.

2. Define a ``gitignore`` Immediately

Don't wait until after the first accidental commit of a 10MB `"coverage.out"`. Define your ``gitignore`` file up front.

Claude can generate a basic one — or you can grab the standard Go template and layer in your own. Either way, making it explicit avoids noise in diffs, bloated repos, and accidental secrets ending up in source control.

3. Define Interfaces, Not Implementations

Want clean code from Claude? Give it clean contracts.

Start by writing interfaces that define your desired behaviors. Claude can then implement those interfaces cleanly, keeping responsibilities focused and reducing coupling. Don't dump implementation and ask for help. Show it the boundaries and let it fill in the middle.

Go's interface-driven design isn't just good practice — it's how you keep Claude producing idiomatic, modular code.

4. Always Start with a Plan (Claude Planning Mode)

Jumping into code without a plan? You're just asking Claude to improvise wildly.

Instead, begin each session by asking Claude to "Plan the steps to implement X." You'll get a bullet list of components, responsibilities, and relationships. Confirm or edit that plan before coding begins. You can also turn on planning mode (Shift+Tab).

Planning mode builds alignment. It's the AI equivalent of writing acceptance criteria before implementation — faster, clearer, and less prone to surprises.

5. If Building an API, Start with OpenAPI

Claude understands OpenAPI/Swagger specs extremely well. Feed it your spec and it can generate:

- Handlers

- Validators

- DTOs

- Test scaffolding

- Client SDKs

With a spec-driven approach, you can regenerate with confidence, keep behavior aligned to contract, and avoid creeping divergence between what you think your API does and what it actually does.

🔧 II. Code Generation Best Practices

6. Use Sub-Agents for Task Isolation

LLMs love doing one thing at a time. So give them one job each.

Break your development workflow into sub-agents:

- One builds the handler

- One generates the tests

- One creates documentation

- One writes benchmarks

This narrows the prompt scope and improves output quality. Just like real engineers, LLMs work better when they're not multitasking.

7. Go Parallel with Sub-Agents

If one agent is good, multiple parallel agents are better.

Claude doesn't get tired. You can spin up several sub-agents in parallel to tackle different parts of the codebase — services, models, routes — and then merge their outputs later.

You'll need to review for overlap or naming collisions, but the speed and concurrency gains are worth it.

8. Let Claude Build the Makefile

A good Makefile is the entry point to your project. Claude's great at building these.

Let it generate a Makefile that includes:

- ``make build``

- ``make test``

Get Davin Hills's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☒ Remember me for faster sign in

- ``make lint``

- ``make cover``

- ``make run``

Standardized commands simplify onboarding and enforce consistent behavior across machines and CI systems.

✅ III. Quality Enforcement and Code Hygiene

9. Use a Strict ``golangci.yml`` for Linting

Claude adapts to rules — if you give it any. Start with a strict ``golangci.yml`` to enforce things like:

- No unused vars

- Cyclomatic complexity limits

- Import order

- Error checking

Then reference those rules in your prompts. For example: "This function violates ``gocyclo`` — rewrite it to pass."

It learns quickly when given sharp feedback.

10. Add Git Pre-Commit Hooks for Lint/Test Enforcement

Don't rely on memory—Automate enforcement.

Claude can generate pre-commit hooks that run:

- Linting

- Format checks

- Unit tests

- Coverage thresholds

If a commit fails a check, it's blocked. This avoids a "push and pray" workflow and ensures code hygiene before review.

11. Commit Early, Commit Often

Claude can be... enthusiastic. Sometimes it solves a problem elegantly. Sometimes it rewrites your whole file for no good reason.

Frequent commits let you:

- Track changes

- Revert easily

- Understand what changed when

Ask Claude to summarize diffs if you're lazy. "Summarize the last 3 changes in 1-line git commit messages" works surprisingly well.

🔍 IV. Code Review and Safety Nets

12. Use Other LLMs to Review Code (second-opinion)

Never trust a single source, especially not an LLM.

That's where *second-opinion* (<https://github.com/dshills/second-opinion>) comes in. It uses a second LLM to review your code before merging. Drop in a commit hash, and it'll return comments, red flags, and suggestions.

Two models are better than one. Especially when one writes and the other critiques.

🔄 V. Testing and Feedback Loop

13. Simplify Test Output (Failures + Coverage Only)

Claude loves verbosity. That's great for brainstorming, terrible for test output.

Ask Claude to run tests with quiet flags and coverage summaries. Strip out green checkmark spam and focus on what matters:

- Which tests failed

- What the coverage is

- Where to fix it

You can even ask Claude to "summarize the failed test output in a readable format" and get a human-ready digest for triage.

🔴 Conclusion: You + Claude = Production-Grade Go

Claude is fast. It's also inconsistent. The key isn't blind trust — it's workflow discipline.

If you treat LLMs like teammates — give them structure, enforce rules, review their work — they'll become the most productive part of your team. Follow this blueprint, and you'll write better Go, faster, with fewer headaches.

*In a future article, I'll explore more about how MCP (Model Context Protocol) servers enable powerful workflows for managing LLM coordination, memory, tool use, and sub-agent orchestration — essential infrastructure if you're serious about building with AI at scale.*

AI + Claude + Go + LLM + Programming +

Some rights reserved

👍 12

💬 4

🔖 1

🔗

🔄

📄

Written by Davin Hills

144 followers · 94 following

30+ years of experience in software engineering and architecture.

Follow

Responses (4)

Write a response

What are your thoughts?

👤 mohamed shakhjahh

Jul 14, 2025

This is a solid guide for integrating AI into Go development. The focus on structuring the project from the start and using clear constraints to guide Claude's output is spot on. Practical tips for creating a reliable workflow.

👍 1

Reply

Ozgur Demir

Nov 25

Also, you can take a look at <https://second.github.io/goarch/>

👍

Reply

Andy Spence

Jun 4

Booleans. Define Interfaces not Implementations. Eh, no

👍

Reply

See all responses

More from Davin Hills

MCP SERVER

Davin Hills · Apr 22, 2025

Building an MCP Server from Scratch

(with a Little Help from GPT)

👍 56

👤 2

Davin Hills · Jul 9

Building KiCadAI: An Evidence-First Toolkit for AI-Assisted PCB...

Why AI-generated circuit boards need deterministic writers, machine-readable...

👍

🔖

Integrating NVIDIA Tesla P40 into a Consumer-Level Computer for...

The NVIDIA Tesla P40, which was once a powerhouse in the realm of server-grade...

👍 8

👤 1

REVIEW MCP SERVER IN GO USING OFFICIAL SDKS

Davin Hills · Oct 6, 2025

Building an MCP Code Review Server in Go Using Official SDKs

A Modern Approach with OpenAI, Anthropic, and Google Model Integrations

👍 11

🔖

See all from Davin Hills

Recommended from Medium

MCP

UX Planet

by Nik Babich · Apr 6

MCP is Dead

Why you should avoid using MCP in Claude Code and what to use instead

👍 5.6K

👤 309

🔖 185

You only have weeks left to vibe code

Michal Malewicz · Jun 29

You only have weeks left to vibe code

Then it's over. You better hurry up!

👍 71K

👤 242

🔖 79

7 CODING PATTERNS I STOLE FROM SENIOR ENGINEERS

In Skill Stuff by CodeByUmar · Jun 8

7 Coding Patterns I Stole From Senior Engineers

Most developers do not become better because they learn more syntax. They...

👍 4.4K

👤 76

🔖 89

10 Must-Have Skills for Claude (and Any Coding Agent) in 2026

unideveloper · Mar 9

10 Must-Have Skills for Claude (and Any Coding Agent) in 2026

The definitive guide to agent skills that change how Claude Code, Cursor, Gemini C...

👍 1.6K

👤 30

🔖 8

The L1 Claude Code Community

I Wasted 6 Months Using Claude Code Wrong. Here Are the 14...

In Towards AI by Mouez Yazidi · Apr 26

I Wasted 6 Months Using Claude Code Wrong. Here Are the 14...

From frustrated beginner to power user: The hidden command ecosystem nobody talks...

👍 4.8K

👤 22

🔖 110

BEYOND RAG: How Google's Open Knowledge Format (OKF) is Replacing the Vector Database

Beyond RAG: How Google's Open Knowledge Format (OKF) is...

In The Code Frontier by Secret Dev · Jul 3

Beyond RAG: How Google's Open Knowledge Format (OKF) is...

For the last three years, the default engineering response to any enterprise AI...

👍 2.9K

👤 58

🔖 34

See more recommendations